

BiteFixes Enterprise Architecture Manual

Volume 12

Platform Runtime Architecture (PRA)

Document ID: BFA-012

Version: 1.0

Status: Draft

Language: English

Architecture Level: Enterprise

Table of Contents

1. Introduction
2. Runtime Philosophy
3. Platform Overview
4. Runtime Components
5. Request Lifecycle
6. Runtime Services
7. Communication Layer
8. Storage Layer
9. Cache Layer

- 10. Queue Layer
- 11. Scheduling Layer
- 12. AI Runtime
- 13. Monitoring
- 14. Logging
- 15. Metrics
- 16. Health Checks
- 17. Scaling
- 18. Disaster Recovery
- 19. Future Runtime

Chapter 1

Introduction

The Platform Runtime defines how BiteFixes operates in production.

It specifies how services communicate, process requests, store information, and recover from failures.

Chapter 2

Runtime Philosophy

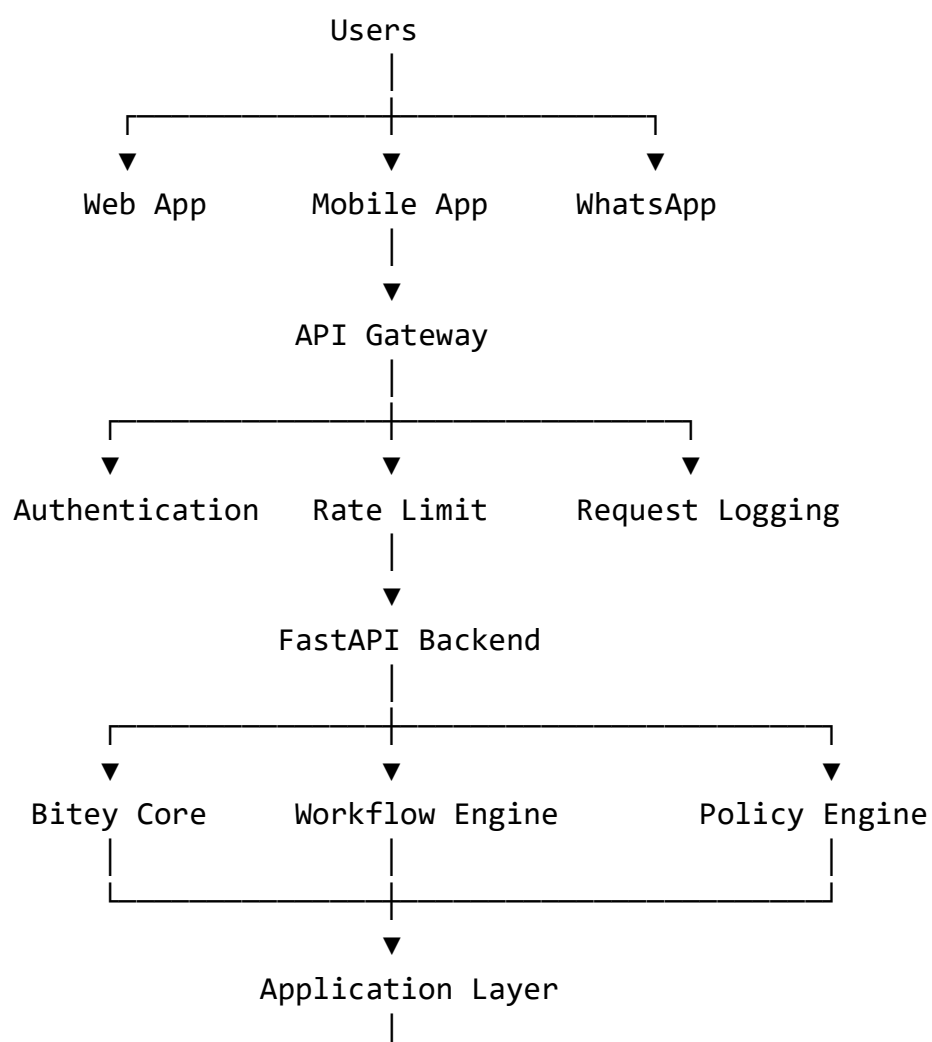
The runtime must satisfy the following goals:

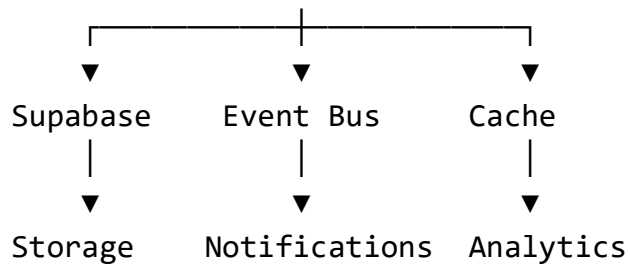
- High availability.
- Fault tolerance.
- Horizontal scalability.
- Security by default.
- Observability.
- Cloud portability.
- Technology independence.

Business logic must remain isolated from infrastructure.

Chapter 3

Platform Overview





Chapter 4

Runtime Components

The platform consists of several independent runtime services.

Edge Layer

- API Gateway
- HTTPS Termination
- Rate Limiter

Security Layer

- Authentication
- Authorization
- Token Validation

Business Layer

- FastAPI
- Application Services
- Domain Services

Intelligence Layer

- Bitey Core

- AI Orchestrator
- Knowledge Retrieval
- Memory Manager

Infrastructure Layer

- PostgreSQL (Supabase)
- Object Storage
- Cache
- Event Bus
- Scheduler

Chapter 5

Request Lifecycle

Every request follows the same path.

Client

↓

HTTPS

↓

API Gateway

↓

Authentication

↓

Authorization

↓

Validation



Application Service



Domain Service



Repository



Database



Event Publication



Response



Logging



Metrics

No request skips security validation.

Chapter 6

Runtime Services

Each service has a single responsibility.

Example:

Customer Service

- Customer registration
- Customer updates
- Customer search

Ticket Service

- Ticket lifecycle
- Assignment
- Status changes

Knowledge Service

- Knowledge search
- Publication
- Approval

Bitey Service

- Intent detection
- Reasoning
- AI orchestration

Chapter 7

Communication Layer

Communication principles:

- REST for synchronous requests.
- Events for asynchronous processing.
- WebSockets for real-time updates.
- Webhooks for external integrations.

No direct database communication between services.

Chapter 8

Storage Layer

The storage layer contains different data categories.

Operational Database

- Companies
- Customers
- Tickets
- Services
- Devices

Knowledge Database

- Knowledge Articles
- AI Prompts
- Embeddings
- Documentation

Audit Database

- Logs
- Security Events
- Workflow History
- AI Decisions

File Storage

- Images
- PDFs
- Invoices
- Device Photos
- Attachments

Chapter 9

Cache Layer

Frequently accessed data is cached.

Examples:

- Company settings
- Permissions
- Active workflows
- Knowledge summaries
- AI prompts

The cache never becomes the source of truth.

Chapter 10

Queue Layer

Background processing uses queues.

Examples:

Email Queue

WhatsApp Queue

Notification Queue

AI Queue

PDF Queue

Import Queue

Export Queue

Queues isolate long-running tasks from user requests.

Chapter 11

Scheduling Layer

Scheduled jobs include:

- Daily backups
- SLA verification
- Notification retries
- AI model synchronization
- Analytics aggregation
- Temporary file cleanup

Schedulers execute independently from user traffic.

Chapter 12

AI Runtime

The AI runtime consists of:

Bitey Core

↓

Memory Manager

↓

Intent Engine

↓

Knowledge Retrieval

↓

Reasoning Engine

↓

Model Orchestrator

↓

LLM Provider

The Model Orchestrator abstracts external AI providers.

Chapter 13

Monitoring

The platform continuously monitors:

- CPU usage.
- Memory usage.
- API latency.
- Database performance.
- Queue depth.
- AI latency.
- Workflow execution.
- Error rates.

Monitoring data feeds operational dashboards.

Chapter 14

Logging

Logs are categorized by type.

- Application logs.
- Security logs.
- AI logs.
- Workflow logs.
- Integration logs.
- Audit logs.

Each log entry includes correlation identifiers for end-to-end tracing.

Chapter 15

Metrics

Key runtime metrics include:

- Requests per second.
- Average response time.
- Error rate.
- Active users.
- AI requests.
- Workflow completions.
- Ticket throughput.
- Cache hit ratio.

Metrics support capacity planning and performance optimization.

Chapter 16

Health Checks

Each service exposes health endpoints.

Typical checks:

- API availability.
- Database connectivity.
- Cache connectivity.
- Queue status.
- AI provider availability.

Health checks support automated orchestration.

Chapter 17

Scaling

Runtime services should scale independently.

Examples:

- Increase API instances during peak hours.
- Scale AI workers based on demand.
- Increase notification workers for bulk messaging.

Stateful components remain centralized where necessary.

Chapter 18

Disaster Recovery

Recovery strategy includes:

- Automated database backups.
- Configuration backups.
- Object storage redundancy.
- Infrastructure as Code.
- Recovery testing.

Recovery objectives should be defined for each environment.

Chapter 19

Future Runtime

The architecture should support future additions without redesign.

Examples:

- Multi-region deployments.
- Kubernetes orchestration.
- AI inference clusters.
- Edge processing.
- Additional cloud providers.
- Hybrid deployments.

The runtime remains modular and portable.

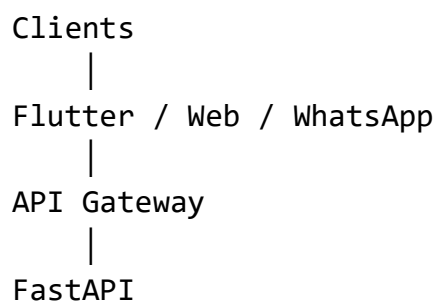
Runtime Design Principles

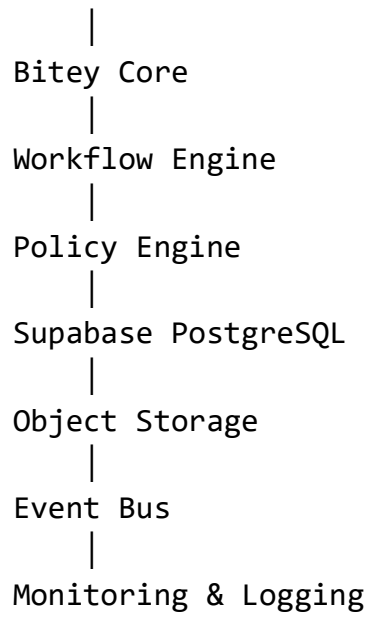
The Platform Runtime follows these principles:

- Stateless application services.
- Independent scaling.
- Observable operations.
- Event-driven communication.
- Secure defaults.
- Cloud-native design.
- Technology abstraction.

Enterprise Runtime Stack (Recommended)

Although the architecture is technology-independent, the current reference implementation for BiteFixes aligns with:





This stack reflects la dirección actual del proyecto, pero cada componente puede sustituirse sin alterar los contratos arquitectónicos definidos en los volúmenes anteriores.